

Unit-III

Arrays

Arrays: Concepts, Using arrays in C:

- An array is a group of related data items that share a common name.
- Ex:- Students
- The complete set of students are represented using an array name students.
- A particular value is indicated by writing a number called index number or subscript in brackets after array name. The complete set of value is referred to as an array, the individual values are called elements.

Accessing the array elements

- **Accessing the array elements:** Accessing the array element is done using a loop statement in combination with printf statements or any other processing statements. Example of accessing the array elements and calculating total marks is given below:

Example:

```
#include<stdio.h>
int main( )
{
int total=0, marks[4]={35,44,55,67};
for(i=0; i<4; i++)
total=total+masks[i]; /* calculating total marks*/
printf("Total marks=%d",total);
return 0;
}
```

Output: Total marks=201

Write a program to search an element using Binary Search

```
.include <stdio.h>
int main()
{
int i, low, high, mid, n, key, array[100];
printf("Enter number of elements:");
scanf("%d",&n);
printf("Enter %d integers", n);
for(i = 0; i < n; i++)
scanf("%d",&array[i]);
printf("Enter value to find");
scanf("%d", &key);
low = 0;
high = n - 1;
mid = (low+high)/2;
while (low <= high) {
if(array[mid] < key)
low = mid + 1;
else if (array[mid] == key) {
printf("%d found at location %d", key, mid+1);
break;
}
else
high = mid - 1;
mid = (low + high)/2;
}
if(low > high)
printf("Not found! %d isn't present in the list", key);
return 0;
}
```

Output:

Enter number of elements:3

Enter 3 integers 1 2 3

1 2 3

Enter value to find 3

3

3 found at location 3

Write Program to sort numbers using Bubble Sort

```
#include <stdio.h>
int main()
{
int array[100], n, i, j, swap;
printf("Enter number of elements\n");
scanf("%d", &n);
printf("Enter %d integers\n", n);
for (i = 0; i < n; i++)
scanf("%d", &array[i]);
for (i = 0 ;i < n - 1; i++)
{for (j = 0 ;j < n - i - 1; j++){
if (array[j] > array[j+1])
{swap = array[j];
array[j] = array[j+1];
array[j+1] = swap;}}}
printf("Sorted list in ascending order:\n");
for (i = 0; i < n; i++)
printf("%d\n", array[i]);
return 0;
}
```

Output:

Enter number of elements

3

Enter 3 integers

5 2 1

Sorted list in ascending order:

1

2

5

ONE – DIMENSIONAL ARRAYS :

- A list of items can be given one variable index is called single subscripted variable or a one-dimensional array.
- The subscript value starts from 0. If we want 5 elements the declaration will be `int number[5];`
- The elements will be `number[0]`, `number[1]`, `number[2]`, `number[3]`, `number[4]` There will not be `number[5]`

Declaration of One - Dimensional Arrays :

Type variable – name [sizes];

Type – data type of all elements Ex: int, float etc., Variable – name – is an identifier

Size – is the maximum no of elements that can be stored. Ex:- float avg[50]

This array is of type float. Its name is avg. and it can contains 50 elements only. The range starting from 0 – 49 elements.

Initialization of Arrays :

Initialization of elements of arrays can be done in same way as ordinary variables are done when they are declared.

Type array name[size] = {List of Value};

Ex:- int number[3]={1,2,3};

Program Showing one dimensional array

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int i;
```

```
float x[10],value,total;
```

```
printf("Enter 10 real numbers\n");
```

```
for(i=0;i<10;i++)
```

```
{
```

```
scanf("%f",&value);
```

```
x[i]=value;
```

```
}
```

```
total=0; for(i=0;i<10;i++)
```

```
total=total+x[i] ;
```

```
for(i=0;i<10;i++)
```

```
printf("x*%2d+=%5.2f\n",i+1,x*i+);
```

```
printf("total=%0.2f",total);
```

```
}
```


TWO – DIMENSIONAL ARRAYS:

- To store tables we need two dimensional arrays. Each table consists of rows and columns. Two dimensional arrays are declare as type array name [row-size][col-size];
- Ex: `int a[10][10];`

INITIALIZING TWO DIMENSIONAL ARRAYS:

They can be initialized by following their declaration with a list of initial values enclosed in braces.

Ex:- `int table[2][3] = {0,0,0,1,1,1};`

Initializes the elements of first row to zero and second row to one. The initialization is done by row by row. The above statement can be written as `int table[2][3] = {{0,0,0},{1,1,1}};`

When all elements are to be initialized to zero, following short-cut method may be used.

`int m[3][5] = {{0},{0},{0}};`

-

Program to find Transpose of a Matrix

```
#include <stdio.h>

int main()
{
    int m, n, c, d, matrix[10][10], transpose[10][10];
    printf("Enter the number of rows and columns of a matrix\n");
    scanf("%d%d", &m, &n);
    printf("Enter elements of the matrix\n");
    for (c = 0; c < m; c++)
        for (d = 0; d < n; d++)
            scanf("%d", &matrix[c][d]);
    for (c = 0; c < m; c++)
        for (d = 0; d < n; d++)
            transpose[d][c] = matrix[c][d];
    printf("Transpose of the matrix:\n");
    for (c = 0; c < n; c++) {
        for (d = 0; d < m; d++)
            printf("%d\t", transpose[c][d]);
        printf("\n");
    }
    return 0;
}
```

Output:

Enter the number of rows and columns of a matrix

2 2

Enter elements of the matrix

2 3 4 5

Transpose of the matrix:

2 4

3 5

Applications of Array

- Data elements can be sorted using arrays. Arrays are used to quickly store and sort elements in a variety of sorting techniques, including Bubble sort, Insertion sort, and Selection sort.
- Matrix operations can be done with arrays. Numerous databases, both small and large, are made up of records-based one- and two-dimensional arrays.

Advantages & Disadvantages of Array

Advantages	Disadvantages
This is a practical method of storing a specified number of similar-type items.	The array can only store a predetermined amount of elements in accordance with the initial size specification. There is no mechanism for expanding the array's size to include additional data down the road, should that be necessary.
Data memory allocation is carried out progressively without using additional memory.	The excess memory space that has already been allocated does not get used if the actual number of elements is fewer than the specified size of an array.
It is quicker to access the arrays by their index value.	Utilizing only one type of data might occasionally have restrictions since, in real-world situations, it may be necessary to use many types of data in array form.
It enables the multidimensional array storage of matrix data items.	It is inconvenient and time-consuming to add or remove records from the array since we must manage memory and indexing.

Functions in C

- A function is a self contained program segment that carries out some specific well defined tasks.
- **Advantages of functions:**
 - 1. Write your code as collections of small functions to make your program modular
 - 2. Structured programming
 - Code easier to debug
 - 4. Easier modification
 - 5. Reusable in other programs

Function Definition

- **Function Definition :**

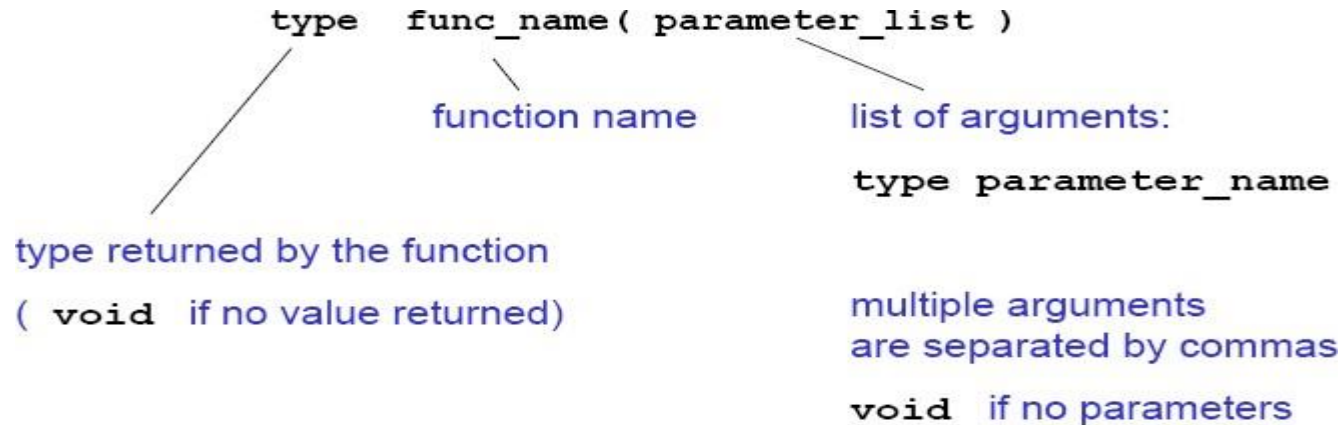
```
type func_name( parameter list )
```

```
{
```

```
declarations; statements;
```

```
}
```


FUNCTION HEADER :



Examples:

```
int fact(int n)
void error_message(int errorcode)
double initial_value(void)
int main(void)
```

Usage:

```
a = fact(13);
error_message(2);
x=initial_value();
```

CATEGORIES OF FUNCTIONS:

- A function depending on whether arguments are present or not and whether a value is returned or not may belong to.
- Functions with no arguments and no return values.
- Functions with arguments and no return values.
- Functions with arguments and return values.
- **Functions with no arguments and no return values :**
- When a function has no arguments, it does not receive any data from calling function.
- In effect, there is no data transfer between calling function and called function.

Arguments but no return values:

- The function takes argument but does not return value.
- The actual (sent through main) and formal(declared in header section) should match in number, type and order.
- In case actual arguments are more than formal arguments, extra actual arguments are discarded. On other hand unmatched formal arguments are initialized to some garbage values.

user-defined function

- A **user-defined function** is a type of function in C language that is defined by the user himself to perform some specific task. It provides code reusability and modularity to our program. User-defined functions are different from built-in functions as their working is specified by the user and no header file is required for their usage.
- Syntax:

```
return_type function_name (type1 arg1, type2  
arg2 .... typeN argN)  
{ // actual statements to be executed // return  
value if any  
}
```

Facctorial of number using Function

```
#include<stdio.h>
#include<math.h>
int main()
{

    printf("Enter a Number to Find Factorial: ");
    printf("\nFactorial of a Given Number is: %d ",fact());
    return 0;
}
int fact()
{
    int i,fact=1,n;
    scanf("%d",&n);
    for(i=1; i<=n; i++)
    {
        fact=fact*i;
    }
    return fact;
}
```

Output:

Enter a Number to Find Factorial: 4

Factorial of a Given Number is: 24

PARAMETER PASSING TECHNIQUES:

- Parameter passing mechanism in C is of two types.
 - Call by Value
 - Call by Reference.
- The process of passing the actual value of variables is known as Call by Value.
- The process of calling a function using pointers to pass the addresses of variables is known as Call by Reference. The function which is called by reference can change the value of the variable used in the call.

Example of Call by Value:

```
#include <stdio.h>
void swap(int,int);
main()
{
int a,b;
printf("Enter the Values of a and b:");
scanf("%d%d",&a,&b);
printf("Before Swapping \n"); printf("a = %d \t b = %d", a,b);
swap(a,b);
printf("After Swapping \n"); printf("a = %d \t b = %d", a,b);
}

void swap(int a, int b)
{
int temp; temp = a; a = b;
b = temp;
}
```

Example of Call by Reference:

```
#include<stdio.h>
main()
{
int a,b; a = 10;
b = 20;
swap (&a, &b); printf("After Swapping \n");
printf("a = %d \t b = %d", a,b);
}
void swap(int *x, int *y)
{
int temp; temp = *x;
*x = *y;
*y = temp;
}
```


RECURSIVE FUNCTIONS:

Recursion is a **repetitive process in which a function calls itself (or) A function** is called recursive if it calls itself either directly or indirectly. In C, all functions can be used recursively.

Example: Fibonacci Numbers

A recursive function for Fibonacci numbers (0,1,1,2,3,5,8,13...)

```
/* Function with recursion*/
```

```
int fibonacci(int n)
```

```
{
```

```
if (n <= 1) return n; else
```

```
return (fibonacci(n-1) + fibonacci(n-2));
```

```
}
```

With recursion 1.4×10^9 function calls needed to find the 43rd Fibonacci number(which has the value 433494437) .If possible, it is better to write iterative functions.

```
int factorial (int n) /* iterative version */
```

```
{
```

```
for ( ; n > 1; --n) product *= n; return product;
```

```
}
```

Factorial of a number using Recursion

```
#include<stdio.h>
```

```
int fact(int);
```

```
int main()
```

```
{
```

```
    int x,n;
```

```
    printf(" Enter the Number to Find Factorial :");
```

```
    scanf("%d",&n);
```

```
    x=fact(n);
```

```
    printf(" Factorial of %d is %d",n,x);
```

```
    return 0;
```

```
}
```

```
int fact(int n)
```

```
{
```

```
    if(n==0)
```

```
        return(1);
```

```
    return(n*fact(n-1));
```

```
}
```

- **Fibonacci Series using Recursion**

```
#include<stdio.h>
```

```
int Fibonacci(int);
```

```
int main()
```

```
{
```

```
    int n, i = 0, c;
```

```
    scanf("%d",&n);
```

```
    printf("Fibonacci series\n");
```

```
    for ( c = 1 ; c <= n ; c++ )
```

```
    {
```

```
        printf("%d\n", Fibonacci(i));
```

```
        i++;
```

```
    }
```

```
    return 0;
```

```
}
```

```
int Fibonacci(int n)
```

```
{
```

```
    if ( n == 0 )
```

```
        return 0;
```

```
    else if ( n == 1 )
```

```
        return 1;
```

```
    else
```

```
        return ( Fibonacci(n-1) + Fibonacci(n-2) );
```

```
}
```

Important Questions

1. Define Array. List out the Advantages and Disadvantages of Array.
2. Explain how do you declare and Initialize an array with syntax and example.
3. List out different types of Array with Example
4. Write a C program to search an element in an array using binary search.
5. Write a C program to search an element in an array using Linear search.
6. Write a program for sorting an element in an array using Bubble Sort
7. Write a C Program for multiplication of 2 matrix.
8. Write a C Program for sum of 2 matrix
9. Write a C program for Transpose of a Matrix.

10. Define Function. Explain the Categories of Function

11. What are different Parameter Passing Techniques. Explain with example program.

12. Define Recursion. Write a program to find factorial of a number using Recursion.

13. Write a c Program to find Fibonacci Series for the given integer input.

14. What would be printed by the following program?

```
#include<stdio.h>
int main()
{
    inti;
    int list[10]={2,1,2,4,1,2,0,2,1,2};
    int line[10];
    for(i=0; i<10; i++)
        line[i] = list[9-i];
    for(i=0; i<10; i++)
        printf("%d %d \n", list[i],line[i] );
    return 0;
}
```